# Day 6 Evidence Workbooklet — Instructor Key

Branch Outcomes with if and else

Engineering practice → condition result → branch followed → selected action → support plan

Instructor key. Same layout as student copy; solutions filled in response boxes. Print format: duplex, left-stapled packet.

| | | | | | |
|---|---|---|---|---|---|
| **Name** | | **Section** | | **Date** | |
| **Score** | _____/42 | | Mastery check | secure / developing / needs support | |

The Week 2 scenario starts with a sensor-kit checkout table. A kit may be released, charged, sent to calibration, or held for missing checkout information. Today the focus is a single branch decision: read one condition, choose the branch that runs, and explain the action selected.

| | |
|---|---|
| **Engineering task** | Decide which checkout action a sensor kit should receive from one condition at a time. |
| **Computational move** | Turn a true/false condition into one selected branch outcome while keeping the evidence for that branch outcome visible. |
| **Python focus** | `if`, `else`, condition result, selected branch, action variable, and printed branch outcome evidence. |
| **Evidence to keep** | case value, condition expression, `True`/`False` result, branch followed, action assigned, and support note. |

## Day 6 working rules

1. Python checks the condition after `if`.
2. If the condition is `True`, the `if` branch runs.
3. If the condition is `False`, the `else` branch runs.
4. Only one of those two branches runs.
5. A reliable branch note names both the condition result and the selected action.

## 1 Read the checkout condition before choosing an action

recognize predict

The lab team is checking whether a sensor kit can leave the checkout table. Day 6 uses one decision at a time: if the condition is true, one action happens; otherwise the other action happens.

```
kit_label = "S-14"
charge_percent = 82
charge_minimum = 80
calibration_label = "current"
borrower_initials = "JL"

charge_ok = charge_percent >= charge_minimum
calibration_ok = calibration_label == "current"
borrower_logged = borrower_initials != ""

if charge_ok:
    action = "send to calibration check"
else:
    action = "charge before checkout"
```

| Pts. | Prompt | My evidence |
|---|---|---|
| 1 | Which variable stores the kit label? | **Key:** `kit_label`. |
| 1 | Which comparison creates `charge_ok`? | **Key:** `charge_percent >= charge_minimum`; here that is `82 >= 80`. |
| 1 | Is `charge_ok` true or false for this kit? Show the numbers. | **Key:** `True`; `82 >= 80`. |
| 1 | Which branch assigns the action for this kit? | **Key:** The `if` branch runs because `charge_ok` is `True`; it assigns `send to calibration check`. |

## 2 Trace an `if/else` branch outcome

trace explain

Complete the branch outcome table. Use the condition result to decide which branch runs.

| Pts. | Charge value | Condition | Branch followed | Action and reason |
|---|---|---|---|---|
| 2 | 82 | `charge_percent >= 80` | **Key:** see reason | **Key:** Result `True`; follow the `if` branch and set action to `send to calibration check` because `82 >= 80`. |
| 2 | 80 | `charge_percent >= 80` | **Key:** see reason | **Key:** Result `True`; follow the `if` branch and set action to `send to calibration check`; equality is included by `>=`. |
| 2 | 79 | `charge_percent >= 80` | **Key:** see reason | **Key:** Result `False`; follow the `else` branch and set action to `charge before checkout` because `79 < 80`. |
| 2 | Explain why the exact value `80` matters. | **Key:** `80` is the boundary. With `>=`, `80 >= 80` is `True`, so the exact minimum follows the `if` branch. | | |

### Boundary connection

The branch outcome depends on a comparison from Week 1. If the comparison is wrong, the selected branch outcome will be wrong too.

## 3 Choose the selected action from printed evidence

predict explain

The same structure is used for calibration status. Read the code and predict the action.

```
calibration_label = "expired"
calibration_ok = calibration_label == "current"

if calibration_ok:
    checkout_status = "ready for borrower log"
else:
    checkout_status = "send to calibration bench"

print(kit_label, checkout_status)
```

| Pts. | Prompt | My evidence |
|---|---|---|
| 2 | What value is stored in `calibration_ok`? Why? | **Key:** `False`, because `calibration_label` is `expired`, not `current`. |
| 2 | Which branch runs: the `if` branch or the `else` branch? | **Key:** The `else` branch runs because `calibration_ok` is `False`. |
| 2 | What exact status text is printed after the kit label? | **Key:** `send to calibration bench`. |
| 2 | Why would `bool(calibration_label)` be unsafe here? | **Key:** Any nonempty label such as `expired` is truthy, so it could look acceptable even though the needed value is exactly `current`. |

## 4 Complete one branch condition

implement debug

Complete the condition so the kit moves to borrower logging only when the borrower initials are present.

```
if ________________________________________________:
    next_step = "record borrower and release kit"
else:
    next_step = "hold until borrower initials are entered"
```

| Pts. | Prompt | My response |
|---|---|---|
| 3 | Write the completed condition using `borrower_initials`. | **Key:** `borrower_initials != ""`. |
| 2 | Why should the condition compare against an empty string instead of checking the kit label? | **Key:** The branch is about whether borrower initials were entered. A kit label can exist even when `borrower_initials` is empty. |
| 2 | Give one input value that should follow the `else` branch. | **Key:** `borrower_initials = ""` should follow the `else` branch. |
| 1 | Name the evidence type you used: state / type / boundary / branch. | **Key:** branch. |

## 5 Debug a branch-outcome claim

debug test explain

A teammate writes: "The kit is ready because `charge_ok` is true." Use branch-outcome evidence to evaluate that claim.

| Pts. | Prompt | My response |
|---|---|---|
| 2 | What part of the claim is supported by the code? | **Key:** The charge evidence is supported: `charge_ok` is `True` because `82 >= 80`. |
| 2 | What evidence is still missing before a full checkout release? | **Key:** Need calibration and borrower evidence, such as `calibration_ok` being `True` and `borrower_initials` not being empty. |
| 2 | Give one test case that should follow the hold or repair outcome. | **Key:** Example: `charge_percent = 82`, `calibration_label = "expired"`, and initials present should produce calibration support, not release. |
| 2 | Write a safer one-sentence checkout note. | **Key:** The kit has enough charge, but release should wait until calibration is `current` and borrower initials are present; otherwise use the matching support plan. |

## 6 Transfer and support plan

transfer explain

| Pts. | My checkout-decision note |
|---|---|
| 4 | Write a short note that says how an `if/else` statement chooses one action from one condition. Use the sensor-kit checkout example.<br>**Key:** An `if/else` statement checks one condition and runs exactly one branch. For charge, `82 >= 80` is `True`, so the `if` branch chooses `send to calibration check`; a low charge would choose `charge before checkout`. |
| 2 | Circle the support plan you would use next: condition result / branch followed / exact boundary / string label / written explanation. Name one next action: _____ |

### Bridge to Day 7

Day 6 chooses between two actions. Day 7 adds ordered rules where the first true condition wins and later branches are skipped.

## 7 Lab bridge: read and call a provided function

VIP 18.8 preparation | Instructor Key

### Mentor scope: use the wrapper; do not author it

Keep the `def` line and parameter names fixed. Ask students to name the input values, calculate the intermediate total, identify the selected branch outcome, and state the exact returned text. The page is unscored.

```
def badge_sheet_first_branch(group_count, badges_per_group):
    total_badges = group_count * badges_per_group
    if total_badges <= 8:
        return "one sheet"
    else:
        return "more than one sheet"
```

| Function call | Total badges | Returned text and branch outcome evidence |
|---|---|---|
| `badge_sheet_first_branch(4, 2)` | **Key:** 8 | **Key:** `"one sheet"`; `8 <= 8` is true, so the `if` branch runs. |
| `badge_sheet_first_branch(3, 3)` | **Key:** 9 | **Key:** `"more than one sheet"`; `9 <= 8` is false, so the `else` branch runs. |
| `badge_sheet_first_branch(2, 4)` | **Key:** 8 | **Key:** `"one sheet"`; the exact boundary value 8 stays on the first branch outcome. |

### Mentor prompt

Ask students to identify which value entered each parameter, what total was stored, and which condition selected the returned text. Do not turn the page into a function-definition lecture.

# Bridge Day 6 VIP Evidence Handoff

INSTRUCTOR KEY • Contract 07a04826c975 • Accessible v5 successor

## Why engineers use this page

Finish the current calculation-and-output lab with top-level code cells; do not use or author a function wrapper.

**Decision boundary:** Code is useful only when its stored state, visible result, assumptions, units, limits, and tests support a defensible engineering interpretation.

## Construct readiness before independent work

**New authoring tools:** list literal as supplied data, aggregate builtin call, len call, round call, multi step model
**Carried authoring tools:** string literal, assignment, print call, exact output, numeric literal, arithmetic expression, input call, int conversion, float conversion, f string
**Supplied-code exposure only:** comparison expression, if elif else
An exposure-only construct may be traced in complete supplied code, but the independent task will not require students to invent it before its authoring day.

## Notebook cells and task constructs

| Activity | Work | Stable cells | Required authoring constructs |
|---|---|---|---|
| **18.8** | Badge Sheet Decision | `18-8-work / 18-8-transfer / 18-8-cold` | arithmetic expression, f string, input call, int conversion |
| **18.9** | Simple Statistics | `18-9-work / 18-9-transfer / 18-9-cold` | aggregate builtin call, f string, len call |
| **18.10** | Box Model Values | `18-10-work / 18-10-transfer / 18-10-cold` | f string, float conversion, input call |
| **18.11** | Structured Text Shape | `18-11-work / 18-11-transfer / 18-11-cold` | profile-level contract |
| **18.12** | Storage Bay Estimate | `18-12-work / 18-12-transfer / 18-12-cold` | f string, float conversion, input call |

## Readiness evidence

1. For Activity 18.8, write the two exact boundary values that must be tested before you open the notebook.

   **Model response:** The exact decision boundaries are 8 and 16. Test values immediately below, at, and immediately above each boundary.

2. Choose one Activity 18.9-18.12 and name the intermediate value and exact displayed result you will inspect first.

   **Model response:** Accept any activity-specific answer that names a real intermediate quantity and its required output, such as total and average for 18.9 or volume and surface area for 18.10.

## Timing and help trigger

Record a first prediction before running. If you still lack an executable plan after about 8 focused minutes, use the linked ThinkCSPy refresher or the supplied model. If two attempts fail for the same named requirement, write the exact mismatch and ask a peer mentor or instructor a targeted question. Timing is diagnostic evidence, not a speed grade. **Start or elapsed minutes:** _____ **My help trigger:** _____

# Engineering Evidence Record

Complete one record from a model, transfer, or Independent Program Studio build. Generic statements are not sufficient; use actual values, a real revision, and one concrete next test.

| Evidence field | Record |
|---|---|
| **Prediction** | A specific expected state or output before execution. |
| **Observed Result** | The actual state or output, including a value or exact text. |
| **Revision Reason** | The defect or mismatch and the change that addresses it. |
| **Engineering Meaning** | How the evidence changes or supports the engineering decision. |
| **Units Or Not Applicable** | The physical unit, or the evidence type when no unit applies. |
| **Assumption** | One condition accepted as true for this model or rule. |
| **Model Limit** | One situation the result does not justify or predict. |
| **Next Test** | A concrete boundary, invalid, or alternate case with an expected result. |
| **Help Trigger** | The exact unresolved point that would trigger a targeted question. |
| **Minutes Used** | Approximate minutes from first prediction through revision. |

## Notebook and submission procedure

1. Attempt, predict, run, inspect, and revise before opening a reveal.
2. Complete the end-of-notebook Independent Program Studio without a reveal or copied solution. Only those visibly labeled Studio cells are scored for independent mastery.
3. Save or download the completed notebook as one `.ipynb` file.
4. Upload that one notebook to the matching Gradescope assignment. Do not export a `.py` file or create a ZIP.